

El Gato | Tic Tac Toe - Enunciado

// 05

Extiende el proyecto

¿Por qué extender?

Los requerimientos anteriores son el piso mínimo. Se espera que cada equipo **proponga y documente** al menos 2 requerimientos adicionales propios que amplíen el sistema de manera coherente. Deben ser aprobados por el docente antes de implementarse y documentados en el README y en Postman.

// IDEAS ORIENTADORAS – NO SON OBLIGATORIAS, SON INSPIRACIÓN

Cualquiera de estas ideas puede adoptarse o modificarse. También puedes proponer algo completamente distinto, siempre que agregue lógica de negocio real.

Abandono de partida

Un jugador puede abandonar una partida activa: `POST /api/games/{gameId}/forfeit` (server). El oponente gana automáticamente. El cliente expone este endpoint vía `POST /api/game/{gameId}/forfeit`. El status pasa a `FINISHED` con el winner siendo el que no abandonó.

Historial de movimientos

El server registra cada movimiento en una entidad `Move` (gameId, playerId, position, symbol, timestamp). Endpoint: `GET /api/games/{gameId}/history`. El cliente expone el historial al jugador autenticado si participa en esa partida. Permite ver la partida como replay.

Ranking global

El server expone `GET /api/players/ranking?top=10` con los jugadores ordenados por victorias o winRate. No requiere autenticación (información pública del juego). El cliente puede re-exponerlo sin filtrar, ya que es un endpoint de consulta pública del dominio del juego, no datos personales del

jugador.

Invitación directa

Al crear la partida, el jugador puede indicar un `invitePlayerId`. La partida queda en estado `WAITING_FOR_INVITE` y solo ese jugador puede unirse. El server valida que el ID del que intenta unirse coincida con el invitado. El cliente expone esta opción al crear la partida vía `serverUrl`.

Modo revancha

`POST /api/games/{gameId}/rematch`: cualquier jugador puede proponer revancha al finalizar. Si el otro acepta (`POST /api/games/{gameId}/rematch/accept`), se crea automáticamente una nueva partida con los mismos jugadores e igual tipo (`NO_TIME/TIMED`), pero con los roles invertidos. La nueva partida retorna en la respuesta.

Health check del server desde el cliente

`GET /api/game/server-health?serverUrl={url}`: el cliente hace un ping al server indicado y retorna si está disponible, la latencia aproximada y la versión del server (si el server la expone). Útil para que el jugador verifique la disponibilidad antes de crear una partida. Siempre con fallback si el server no responde.

Regla de extensión: Las features propias deben implementarse con el mismo estándar que las obligatorias: endpoints REST nivel 2, validaciones, códigos HTTP correctos, fallback si aplica, y documentación en Postman. No se aceptan features sin endpoints funcionales.

← 04 Estándares

06 Entrega →

